



# CJC

## Complete Java Classes

by Kunal Sir

**Q1.** Define an interface called Shape that declares a method named calculateArea(). Then create the classes Circle, Rectangle, and Triangle that implement this interface. Each class should provide its logic to compute the area specific to its geometric shape (**for instance, the area of a circle can be calculated using the formula  $\pi * \text{radius}^2$ , while the area of a rectangle is determined by multiplying its length by its width**). Finally, in the main method, create 3 shape objects and call their respective methods.

**Q2.** define an interface named Payment that contains a method called processPayment(double amount). Next, create specific classes for each payment method, such as GPayPayment, PhonePayPayment, and PaytmPayment. Each of these classes should implement the process payment method, providing an appropriate output message that indicates the payment method being utilized (for example, "Processing payment via PhonePay" or "Processing payment via GPay"). Finally, in the main method, demonstrate the payment processing by instantiating these payment classes and invoking their respective processPayment methods to simulate transactions.



# CJC

## Complete Java Classes

by Kunal Sir

Q3. Define an interface called `Sim` with methods such as `makeCall(String number)`, `sendText(String message)`, and `useData(double dataAmount)`. Next, create another interface called `Consumer`, which should include methods like `useSim(Sim sim)` and `getCustomerDetails()`. Then, implement various types of SIM cards by creating classes like `PrepaidSim` and `PostpaidSim`, both of which will implement the `Sim` interface, providing specific functionalities for making calls, sending texts, and using data. Additionally, create different consumer classes such as `NormalCustomer` and `PremiumCustomer` that will implement the `Consumer` interface and define how these consumers interact with the SIM cards. and In the `main` method, simulate scenarios where these customers are using different types of SIM cards to perform actions like making calls, sending messages, and using data.

// Sim Interface

```
interface Sim {  
  
    void makeCall(String number);  
  
    void sendText(String message);  
  
    void useData(double dataAmount);  
  
}
```



# CJC

Complete Java Classes

by Kunal Sir

// Consumer Interface

```
interface Consumer {  
  
    void useSim(Sim sim);  
  
    String getCustomerDetails();  
  
}
```

// PrepaidSim Class - Implements Sim Interface

```
class PrepaidSim implements Sim {  
  
    public void makeCall(String number) {  
  
        System.out.println("Making a call to " + number + " using Prepaid SIM");  
  
    }  
  
    public void sendText(String message) {  
  
        System.out.println("Sending a text: " + message + " using Prepaid SIM");  
  
    }  
  
}
```



# CJC

## Complete Java Classes

by Kunal Sir

```
public void useData(double dataAmount) {  
  
    System.out.println("Using " + dataAmount + "MB data with Prepaid SIM");  
  
}
```

// PostpaidSim Class - Implements Sim Interface

```
class PostpaidSim implements Sim {  
  
    public void makeCall(String number) {  
  
        System.out.println("Making a call to " + number + " using Postpaid SIM");  
  
    }
```

```
    public void sendText(String message) {  
  
        System.out.println("Sending a text: " + message + " using Postpaid SIM");  
  
    }
```

```
    public void useData(double dataAmount) {  
  
        System.out.println("Using " + dataAmount + "MB data with Postpaid SIM");  
  
    }
```



# CJC

Complete Java Classes

by Kunal Sir

```
}  
  
}  
  
// RegularCustomer Class - Implements Consumer Interface
```

```
class NormalCustomer implements Consumer {
```

```
    private String name;
```

```
    public RegularCustomer(String name) {
```

```
        this.name = name;
```

```
    }
```

```
    public void useSim(Sim sim) {
```

```
        System.out.println(name + " is using the SIM card.");
```

```
        sim.makeCall("1234567890");
```

```
        sim.sendText("Hello, how are you?");
```

```
        sim.useData(100); // Example data usage
```

```
    }
```



# CJC

Complete Java Classes

by Kunal Sir

```
public String getCustomerDetails() {  
  
    return "Regular Customer: " + name;  
  
}  
  
}  
  
// PremiumCustomer Class - Implements Consumer Interface  
class PremiumCustomer implements Consumer {  
  
    private String name;  
  
    public PremiumCustomer(String name) {  
  
        this.name = name;  
  
    }  
  
    public void useSim(Sim sim) {  
  
        System.out.println(name + " is using the SIM card.");  
  
        sim.makeCall("8787878787");  
  
    }  
  
}
```



# CJC

## Complete Java Classes

by Kunal Sir

```
sim.sendText("I'm doing great, thank you!");
```

```
sim.useData(500); // Example data usage
```

```
}
```

```
public String getCustomerDetails() {
```

```
    return "Premium Customer: " + name;
```

```
}
```

```
}
```

```
// Main Class to Demonstrate the Task
```

```
public class TelecomApp {
```

```
    public static void main(String[] args) {
```

```
        Sim prepaidSim = new PrepaidSim();
```

```
        Sim postpaidSim = new PostpaidSim();
```

```
        Consumer normalCustomer = new NormalCustomer("Ajay");
```

```
        Consumer premiumCustomer = new PremiumCustomer("Vijay");
```



# CJC

Complete Java Classes

by Kunal Sir

```
System.out.println(regularCustomer.getCustomerDetails());
```

```
regularCustomer.useSim(prepaidSim);
```

```
System.out.println("\n" + premiumCustomer.getCustomerDetails());
```

```
premiumCustomer.useSim(postpaidSim);
```

```
}
```

```
}
```

**Q3. Design an interface called Mover, which contains a method named moveItem(String item, String destination). Additionally, create another interface called Packer that includes a method called packItem(String item, String boxType). then implement classes such as FurnitureMover, ElectronicsPacker, and FullServiceWorker, where each class implements the relevant interface(s). For instance, FurnitureMover should implement the Mover interface, while ElectronicsPacker should implement the Packer interface. The FullServiceWorker class can implement both interfaces. And In the main method, instantiate these classes and simulate various moving and packing tasks by invoking their respective methods.**





# CJC

**Complete Java Classes**

**by Kunal Sir**



# CJC

**Complete Java Classes**